Architecture and Protocols – KampüsAjanda

MTM4681 Mobile Application Development · Milestone · Elif Ağpınar, Zeynep Yüksel, Nisa Ay

1. Problem, Users and Evidence

Users: YTÜ students with busy class schedules, and student clubs that organize campus events.

Problem: Campus events are announced on many different channels, and none of them knows a student's timetable. Students often hear about an event too late, or find out that it overlaps with a class.

Evidence: We ran a short survey with 14 YTÜ students.

- 13 of 14 students (93%) missed at least one campus event in the last month because they heard about it too late.
- 64% said the main reason they cannot attend events is a conflict with their classes.
- Students follow events on Instagram (71%), through friends (50%) and in WhatsApp groups (43%), so the information is scattered.
- 71% rated a schedule-based event app 4 or 5 out of 5, and 79% prefer a reminder one day before an event.

> *"An app that follows my interests and my timetable and tells me about current events that fit my free time." – survey participant (translated from Turkish)*

Why existing apps are not enough: Instagram and WhatsApp groups are scattered and do not know the student's schedule; the university website is static; Google Calendar does not know campus events; ticketing apps such as Biletix do not focus on student clubs.

2. Core Features and Course Topics

#	Feature	Screens	Course topics
1	Login and registration with school e-mail; every user registers as a student	1, 2, 3	Firebase Authentication, navigation
2	Onboarding: at least 3 interests and the weekly timetable	4, 5	Local DB (Hive), MVVM
3	"For You" list ranked by free time and interests, with a short reason on each card	6, 6.1, 13	Firestore (real-time), MVVM, responsive layout
4	Event detail, joining and a 1-day reminder; joined events appear in the calendar	7, 8	Navigation, animation (card → detail), local notifications
5	Weather for outdoor events	7.1	API/JSON (Open-Meteo REST API)
6	Club panel: become an admin with an SKS code, create and manage events, manage up to 3 admins	9.1, 10, 10.1, 11, 12	Firebase, navigation, form validation and error handling

3. Design Pattern: MVVM with Riverpod

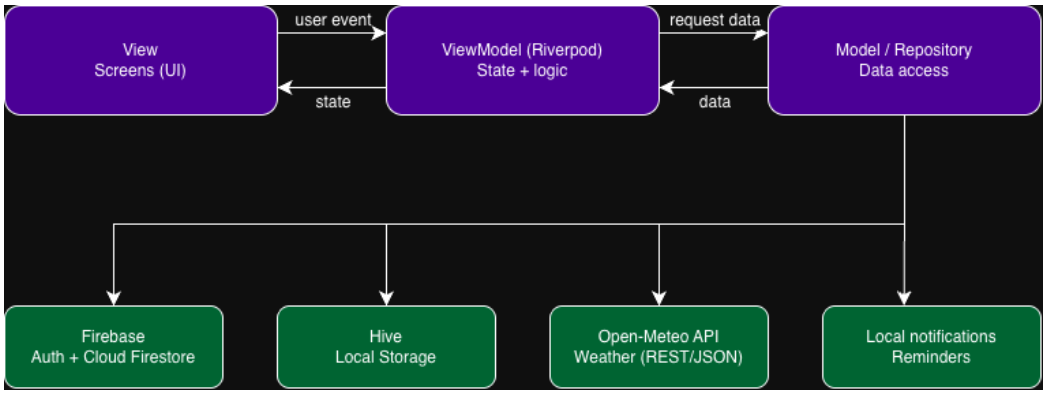


Figure 1. MVVM architecture of KampüsAjanda

Layer	Responsibility	Example
View	Draws the screen and sends user events (taps, form input) to the ViewModel. Contains no business logic.	`ForYouScreen`, `EventDetailScreen`
ViewModel	Holds the screen state (loading, data, error), calls the repositories and the matching service, and exposes the result to the View with Riverpod.	`ForYouViewModel` → ranked list and "why recommended" reasons
Model / Repository	Data classes and repositories. A repository decides where the data comes from (Firestore, Hive or the API); the ViewModel does not know.	`EventRepository`, `ScheduleRepository`

Example flow: The student opens "For You" → `ForYouViewModel` listens to `EventRepository.watchEvents()` and reads the timetable from `ScheduleRepository` → the matching service scores each event → the ViewModel publishes the ranked list as state → the View redraws

the cards. When a club publishes a new event, Firestore pushes it and the list updates without a refresh.

Why Riverpod instead of Provider: Riverpod catches more errors at compile time, while we are writing the code; with Provider some of these errors only appear when the app is running, which can cause crashes in front of the user. Riverpod also does not depend on the widget tree (BuildContext), so ViewModels are easier to test, and `StreamProvider` makes it simple to listen to Firestore in real time.

4. Screen Map and Navigation

The complete screen map with all decisions is in **2_user_flow.pdf**. Navigation uses **go_router**: users who are not logged in are redirected to the Welcome screen (01); the student side has a bottom navigation bar (For You · Calendar · Profile); the club panel (10–12) is opened from Profile and only shown to club admins; 9.1 and 10.1 open as bottom sheets.

Group	Screens
Onboarding	01 Welcome · 02 Log In · 03 Sign Up · 04 Interests · 05 Timetable
Student	06 For You (06.1 offline) · 07 Event Detail (07.1 outdoor) · 08 Calendar · 09 Profile · 13 Tablet
Club	09.1 Activate Club Account · 10 Club Panel · 10.1 Admins · 11 Create Event · 12 My Events

5. Data Model and Firestore Structure

The fields were derived from the information shown on our final mockups.

Model	Main fields
Event	id, title, description, clubId, clubName, posterUrl?, startTime, endTime, location, tags, capacity, attendeeCount, isOutdoor
UserProfile	uid, name, email, interests, reminderMinutes (default 1440 = 1 day), adminClubId?
Club	id, name, isVerified, presidentUid, adminUids (max 3)
ClubCode	code (8 characters), clubId, isUsed, expiresAt (30 days)
CourseSlot	day (1–5), startHour, endHour
Attendance	eventId, joinedAt, reminderOn

```
users/{uid}            → UserProfile
  courses/{slotId}     → CourseSlot
  attending/{eventId}  → Attendance
clubs/{clubId}         → Club
clubCodes/{code}       → ClubCode
events/{eventId}       → Event
```

Not stored: *FreeSlot* (gaps between lectures, calculated on the device from CourseSlots) and *WeatherInfo* (temperature and rain probability, fetched from Open-Meteo because the forecast changes).

Screen	Calculated value	Calculated from
6, 7	Match score and "Why recommended?"	FreeSlots, interests, attendeeCount / capacity (section 7)
9	Joined events, free time this week	Number of Attendance records; sum of FreeSlots
10	Active events, total attendance	The club's events
11	Estimated reach	A small `stats/interestCounts` document keeps the number of students for each interest (updated by ±1 when a student saves their interests). The estimate is the highest count among the event's tags, so other users' documents are never read.

Design decisions: There are no separate club accounts: club admins use their own student account, so no password is ever shared and the club can be handed over to a new president. Joined events and timetables are sub-collections of each user, so every user only reads and writes their own data. The only shared values students change are counters (attendeeCount and interestCounts), and security rules allow them to change these only by 1. Calculated values are not stored, so they never get out of sync.

6. Protocols and Interfaces

Protocol / service	Purpose in KampüsAjanda	Screens
HTTPS + REST/JSON (Open-Meteo)	Gets the weather forecast for outdoor events with a GET request and parses the JSON response. No API key is needed.	7.1
Firebase Authentication	Registration and login with school e-mail and password; the app only accepts `@std.yildiz.edu.tr` addresses (checked in the app) and e-mail verification is required. Keeps the user logged in.	2, 3
Cloud Firestore	Cloud database for users, clubs, codes and events. Real-time listeners update the "For You" list when a club publishes an event. Security rules: users write only their own documents; only club admins create or cancel events; a student may only increase an event's `attendeeCount` by 1 when joining, in the same transaction that creates their Attendance record.	4–12
Hive (local database)	Saves the timetable and the last loaded events on the phone, so the app works offline.	5, 6.1, 8
Local notifications	Sends the reminder one day before a joined event; for outdoor events the rain probability at the time of joining is added to the message.	7, 9

Example request:

```
GET https://api.open-meteo.com/v1/forecast?latitude=41.02&longitude=28.89
    &hourly=temperature_2m,precipitation_probability&timezone=Europe/Istanbul
```

The app finds the event's hour in the `hourly.time` list and reads the temperature and rain probability at the same index (shown as "16°C · Rain 10%"). `timezone=Europe/Istanbul` makes the hours match Turkish time.

Repository interfaces (ViewModels depend only on these):

```
abstract class AuthRepository {
  Future<UserProfile> register(String name, String email, String password);
  Future<UserProfile> login(String email, String password);
  Future<void> logout();
}
abstract class EventRepository {
  Stream<List<Event>> watchEvents();
  Future<void> attend(String eventId, bool reminderOn);
  Future<void> createEvent(Event event);
  Future<void> cancelEvent(String eventId);
}
abstract class ScheduleRepository {
  Future<List<CourseSlot>> getSchedule();
  Future<void> saveSchedule(List<CourseSlot> slots);
}
abstract class ClubRepository {
  Future<Club> activateWithCode(String code);
  Future<void> addAdmin(String clubId, String email);
  Future<void> removeAdmin(String clubId, String uid);
}
abstract class WeatherRepository {
  Future<WeatherInfo> getForecast(DateTime time);
}
```

7. Matching Algorithm

Each upcoming event gets a match score between 0 and 100:

```
score = 100 × (0.50 × time + 0.35 × interest + 0.15 × popularity)

time       = 1 if the event fits completely inside a free slot, 0.5 if it partly overlaps a class, 0 if it fully overlaps
interest   = (event tags that are in the student's interests) / (number of event tags)
popularity = attendeeCount / capacity
```

Why these weights: In our survey, 64% of students said class conflicts are the main reason they miss events, so time has the highest weight: an event the student cannot attend is not a useful recommendation, even if the topic is interesting. Topic was the most important choice factor for students who are free (43%), so interest has the second-highest weight. Popularity has a small weight and only separates events that are otherwise similar.

Example: Flutter Workshop, 12:15–13:30 on Tuesday; the student is free 12:00–14:00 (time = 1); tags Technology and Career, both in the student's interests (interest = 1); 34 of 40 places taken (popularity = 0.85) → score = 100 × (0.50 + 0.35 + 0.13) ≈ 98. The parts of the score also produce the "Why recommended?" lines: "Fits your 12:00–14:00 free time", "Matches your interest Technology", "34 people are going". The percentages in the mockups are example values.

8. Error Handling and Offline Mode

Situation	What the user sees
No internet	Warning banner on "For You" (06.1); events loaded earlier are shown from Hive and the Firestore cache
Wrong e-mail or password	Error message under the password field; the user stays on 02
Invalid, used or expired club code	"This code is invalid or has already been used" on 9.1
Missing field when creating an event	The field is marked and "Publish" does not continue
Weather API does not respond (5 s timeout)	The weather row is hidden; the rest of the screen works
Event is more than 16 days away (Open-Meteo forecast limit)	The weather row is hidden until the forecast is available

9. Responsive Design

A `LayoutBuilder` checks the screen width. Below 840 dp (logical pixels) the phone layout is used (one column, bottom navigation bar). From 840 dp, for example a tablet in landscape (13), a `NavigationRail` replaces the bottom bar and the event list and the event detail are shown side by side.

10. Weekly Plan and Risks

Each member builds their own features end to end (View, ViewModel and Repository), so the workload is equal and everyone can explain all three MVVM layers in the Q&A.

Week	Elif: login, onboarding, profile	Zeynep: student main flow	Nisa: club flow and infrastructure
12–15 Nov	Setup: Flutter SDK, Android Studio, emulator and a real phone for everyone; Firebase project (free Spark plan)		
8	Screens 1–3, Firebase Authentication	Theme and shared widgets	Project skeleton (MVVM folders, Riverpod, go_router, all packages), Firebase connection, data models
9	Screens 4–5, timetable with Hive and Firestore	Screen 6, live event list	Screen 11 (create event, validation), demo data
10	Free-time calculation and unit test	Screens 7 and 7.1, joining, card → detail animation	Matching algorithm and unit test, screen 12
11	Screen 9, 1-day reminder notifications	Screen 8 (calendar), Open-Meteo weather	Screens 9.1, 10 and 10.1 (club code, admins)
12	Offline mode (06.1), error messages; README by 20 Dec	Tablet layout (13); demo videos by 20 Dec	Firestore security rules, error messages; APK by 20 Dec
13	Presentation and final testing, peer contribution form, submission		

Code freeze: 18 December. After this date only bug fixes are made; demo videos and screenshots are taken from the frozen version.

Risk	Effect	Solution
No team member has used Firebase before	Login and data features may be delayed	Firebase is set up first (before week 8); official FlutterFire guides are followed
Midterms and other courses reduce working time	Weekly tasks may slip	Small weekly tasks; code freeze on 18 December leaves a buffer before the demo
Matching algorithm is harder than expected	"For You" list may not work well	The first version uses the simple formula in section 7 and is tested with unit tests
Merge conflicts when three people edit the same files	Lost work	Everyone works in their own feature folder; only Nisa edits `pubspec.yaml` and the router
Firebase Storage requires a paid plan	Event posters cannot be uploaded	Posters are optional; events without a poster show a colored letter card
Not enough real event data for the demo	Empty screens in the demo	Sample events of real YTÜ clubs are added to Firestore before the demo